

Brute Force - Searching and String Matching

Sequential Search, Brute Force String Matching

Laboratory Record

Name: **J Karthik**

Register Number: **192472136**

1. Sequential Search

Q1. Sequential Search — Successful Search

Aim

To implement Sequential (Linear) Search to find a given element in an array and count the number of comparisons made.

Algorithm

1. Start.
2. Read the array and the key element to be searched.
3. Set $i = 0$ and comparisons = 0.
4. Repeat while $i < n$: increment comparisons; if array[i] equals key, return position (i+1) and comparisons.
5. Increment i and continue.
6. If the loop ends without a match, report 'Element not found'.
7. Stop.

Python Program

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [12, 25, 8, 45, 32, 19, 50]
key = 32
position, comparisons = sequential_search(arr, key)
print(f"Element found at position {position}")
print(f"Number of comparisons = {comparisons}")
```

Input

```
arr = [12, 25, 8, 45, 32, 19, 50]
key = 32
```

Output

```
Element found at position 5
Number of comparisons = 5
```

Result

The key 32 is found at position 5, and since sequential search compares elements one by one from the start, exactly 5 comparisons are made before the match - confirming $O(i)$ comparisons for a match at position i.

Q2. Sequential Search — Unsuccessful Search

Aim

To implement Sequential Search for the case where the key is not present in the array, and to count the number of comparisons.

Algorithm

8. Start.
9. Read the array and the key element.
10. Set $i = 0$ and $\text{comparisons} = 0$.
11. Repeat while $i < n$: increment comparisons; if $\text{array}[i]$ equals key, return the position.
12. Increment i and continue scanning.
13. If the entire array is scanned with no match, report 'Element not found' along with the comparison count.
14. Stop.

Python Program

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [5, 10, 15, 20, 25, 30, 35]
key = 18
position, comparisons = sequential_search(arr, key)
if position == -1:
    print("Element not found")
print(f'Number of comparisons = {comparisons}')
```

Input

```
arr = [5, 10, 15, 20, 25, 30, 35]
key = 18
```

Output

```
Element not found
Number of comparisons = 7
```

Result

Since 18 is absent from the array, every one of the $n = 7$ elements must be compared before concluding the search has failed, demonstrating the $O(n)$ worst-case behaviour of sequential search for unsuccessful searches.

Q3. First Occurrence Using Sequential Search

Aim

To find the first occurrence of a given (possibly repeated) element in an array using sequential search.

Algorithm

15. Start.
16. Read the array and the key element.
17. Set $i = 0$.
18. Repeat while $i < n$: if array[i] equals key, return position (i+1) immediately and stop scanning.
19. Increment i and continue if no match yet.
20. If the array is exhausted, report 'Element not found'.
21. Stop.

Python Program

```
def first_occurrence(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i + 1  
    return -1  
  
arr = [10, 25, 15, 25, 30, 25, 40]  
key = 25  
position = first_occurrence(arr, key)  
print(f'First occurrence at position {position}')
```

Input

```
arr = [10, 25, 15, 25, 30, 25, 40]  
key = 25
```

Output

First occurrence at position 2

Result

The search stops as soon as the first match is found (at position 2), avoiding unnecessary scanning of the remaining duplicates further in the array - an $O(n)$ worst case but often faster in practice.

Q4. All Occurrences Using Sequential Search

Aim

To find all occurrences (positions) of a given element in an array using sequential search, without stopping at the first match.

Algorithm

22. Start.
23. Read the array and the key element.
24. Initialize an empty list to store matching positions.
25. For $i = 0$ to $n-1$: if $\text{array}[i]$ equals key, append position $(i+1)$ to the list.
26. Continue scanning through the entire array (do not stop early).
27. Display all recorded positions and the total occurrence count.
28. Stop.

Python Program

```
def all_occurrences(arr, key):  
    return [i + 1 for i in range(len(arr)) if arr[i] == key]
```

```
arr = [7, 12, 7, 25, 18, 7, 30, 7]  
key = 7  
positions = all_occurrences(arr, key)  
print("Occurrences at positions:")  
print(", ".join(map(str, positions)))  
print(f"Total occurrences = {len(positions)}")
```

Input

```
arr = [7, 12, 7, 25, 18, 7, 30, 7]  
key = 7
```

Output

```
Occurrences at positions:  
1, 3, 6, 8  
Total occurrences = 4
```

Result

Unlike first-occurrence search, this variant must scan the full array ($O(n)$ always, no early exit), correctly reporting all 4 positions where the key 7 appears.

Q5. Comparisons, Matches and Mismatches Count

Aim

To count the total comparisons, total matches and total mismatches performed while sequentially searching for a key in an array.

Algorithm

29. Start.
30. Read the array and the key element.
31. Initialize comparisons = matches = mismatches = 0.
32. For each element in the array: increment comparisons.
33. If the element equals the key, increment matches; else increment mismatches.
34. Display the final counts after scanning the whole array.
35. Stop.

Python Program

```
def seq_search_stats(arr, key):
    comparisons = matches = mismatches = 0
    for x in arr:
        comparisons += 1
        if x == key:
            matches += 1
        else:
            mismatches += 1
    return comparisons, matches, mismatches

arr = [3, 6, 9, 12, 15, 18, 21]
key = 15
comparisons, matches, mismatches = seq_search_stats(arr, key)
print(f"Total comparisons = {comparisons}")
print(f"Total matches = {matches}")
print(f"Total mismatches = {mismatches}")
```

Input

```
arr = [3, 6, 9, 12, 15, 18, 21]
key = 15
```

Output

```
Total comparisons = 7
Total matches = 1
Total mismatches = 6
```

Result

Scanning the complete array of 7 elements gives 7 comparisons, exactly 1 match (the key itself) and 6 mismatches, verifying that comparisons = matches + mismatches always holds for a full-array scan.

Q6. Sentinel Sequential Search

Aim

To implement Sentinel Sequential Search, which removes the explicit bounds check ($i < n$) from the loop condition, and to compare it with ordinary sequential search.

Algorithm

36. Start.
37. Read the array and the key element.
38. Save the last element of the array, then temporarily replace it with the key (the 'sentinel').
39. Set $i = 0$ and comparisons = 0.
40. Repeat: increment comparisons; while array[i] is not equal to key, increment i.
41. Restore the last element; if i is less than $n-1$, or the original last element equalled the key, the search succeeded at position (i+1).
42. Otherwise report 'Element not found'.
43. Stop.

Python Program

```
def sentinel_search(arr, key):
    a = arr.copy()
    n = len(a)
    last = a[n - 1]
    a[n - 1] = key
    i = 0
    comparisons = 0
    while a[i] != key:
        comparisons += 1
        i += 1
    comparisons += 1
    if i < n - 1 or last == key:
        return i + 1, comparisons
    return -1, comparisons

arr = [14, 9, 22, 35, 18, 41, 27]
key = 18
position, comparisons = sentinel_search(arr, key)
print(f'Position found = {position}')
print(f'Comparison count = {comparisons}')
```

Input

```
arr = [14, 9, 22, 35, 18, 41, 27]
key = 18
```

Output

```
Position found = 5
Comparison count = 5
```

Result

By placing the key as a sentinel at the last index, the algorithm avoids a separate bounds check ($i < n$) on every iteration, reducing per-iteration overhead while still performing the same $O(n)$ worst-case number of comparisons as ordinary sequential search.

Q7. Sequential Search on Register Numbers

Aim

To perform sequential search on an array of student register numbers to locate a given register number.

Algorithm

44. Start.
45. Read the array of register numbers and the search key.
46. Set $i = 0$.
47. Repeat while $i < n$: if register_numbers[i] equals key, return position (i+1).
48. Increment i and continue.
49. If not found after scanning all entries, report 'Register Number not found'.
50. Stop.

Python Program

```
def sequential_search_simple(arr, key):
    for i, x in enumerate(arr):
        if x == key:
            return i + 1
    return -1

reg_numbers = [101, 102, 103, 104, 105, 106]
key = 104
position = sequential_search_simple(reg_numbers, key)
print(f'Register Number found at position {position}')
```

Input

```
reg_numbers = [101, 102, 103, 104, 105, 106]
key = 104
```

Output

Register Number found at position 4

Result

The same sequential-search logic applies directly to register-number data, locating key 104 at position 4 after 4 comparisons, illustrating that the technique is data-type independent as long as equality comparison is defined.

Q8. Sequential Search on String Data

Aim

To implement Sequential Search over an array of names (string data) to locate a given name.

Algorithm

51. Start.
52. Read the array of names and the name to search for.
53. Set $i = 0$.
54. Repeat while $i < n$: if `names[i]` equals the search name (string comparison), return position ($i+1$).
55. Increment i and continue.
56. If not found, report 'Name not found'.
57. Stop.

Python Program

```
def sequential_search_simple(arr, key):
    for i, x in enumerate(arr):
        if x == key:
            return i + 1
    return -1

names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
search_name = "Deepa"
position = sequential_search_simple(names, search_name)
print(f'Name found at position {position}')
```

Input

```
names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
search_name = "Deepa"
```

Output

Name found at position 4

Result

Sequential search on strings works identically to numeric search, using Python's built-in string equality comparison; "Deepa" is correctly located at position 4.

Q9. Sequential Search on a 2-D Matrix

Aim

To perform sequential search on a 2-D matrix to locate a given element and report its row and column.

Algorithm

58. Start.
59. Read the matrix and the key element.
60. For each row r from 0 to $\text{rows}-1$: for each column c from 0 to $\text{cols}-1$: if $\text{matrix}[r][c]$ equals key, return (row $r+1$, column $c+1$).
61. Continue scanning row by row, column by column, until a match is found.
62. If the entire matrix is scanned with no match, report 'Element not found'.
63. Stop.

Python Program

```
def search_2d(matrix, key):
    for r in range(len(matrix)):
        for c in range(len(matrix[0])):
            if matrix[r][c] == key:
                return r + 1, c + 1
    return -1, -1

matrix = [
    [12, 8, 15],
    [5, 18, 27],
    [9, 11, 24],
]
key = 24
row, col = search_2d(matrix, key)
print(f'Element found at Row {row} Column {col}')
```

Input

```
matrix = [[12,8,15],[5,18,27],[9,11,24]]
key = 24
```

Output

Element found at Row 3 Column 3

Result

Treating the matrix as a sequence of rows scanned left-to-right, top-to-bottom, sequential search correctly locates 24 at Row 3, Column 3 after 9 comparisons in the worst case, i.e. $O(\text{rows} \times \text{cols})$.

Q10. Sequential Search — Performance Analysis

Aim

To implement Sequential Search on a 15-element array for multiple search keys, showing every comparison, and to analyze best-case, average-case and worst-case complexity.

Algorithm

64. Start.
65. Read the array of 15 elements.
66. For each search key: set comparisons = 0.
67. Repeat while $i < n$: increment comparisons; if array[i] equals key, return position and comparisons.
68. If the array is exhausted, report 'not found' with the total comparisons made ($= n$).
69. Repeat for every search key and tabulate comparisons for each.
70. Identify the best case (match at position 1), worst case (match at last position or absent) and average case (match at a random position, approx. $(n+1)/2$ comparisons).
71. State the best-case, average-case, worst-case and space complexity.
72. Stop.

Python Program

```
def sequential_search(arr, key):
    comparisons = 0
    for i, x in enumerate(arr):
        comparisons += 1
        if x == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
keys = [73, 18, 100]
for key in keys:
    position, comparisons = sequential_search(arr, key)
    if position != -1:
        print(f"Key {key}: found at position {position}, comparisons = {comparisons}")
    else:
        print(f"Key {key}: not found, comparisons = {comparisons}")
```

Input

```
arr = [45,23,67,12,89,34,56,78,90,11,29,73,18,64,37] (n = 15)
keys = 73, 18, 100
```

Output

```
Key 73: found at position 12, comparisons = 12
Key 18: found at position 13, comparisons = 13
Key 100: not found, comparisons = 15
```

Result

Best-case complexity is $\Omega(1)$ (key at position 1), worst-case complexity is $O(n)$ (key at the last position or absent, as seen for key 100 requiring all 15 comparisons), average-case complexity is $\Theta(n)$ (approximately $(n+1)/2$ comparisons on average), and space complexity is $O(1)$ since no extra memory is used beyond the input array.

2. Brute Force String Matching

Q11. Brute Force String Matching — Pattern Occurrences

Aim

To implement the Brute Force String Matching algorithm to find all positions where a pattern occurs in a given text and to count total character comparisons.

Algorithm

73. Start.
74. Read the text (length n) and the pattern (length m).
75. For each starting position i from 0 to $n-m$: compare $\text{pattern}[0..m-1]$ against $\text{text}[i..i+m-1]$ character by character.
76. Increment the comparison counter for every character compared.
77. If a mismatch occurs, stop comparing at this position and move to $i+1$ (shift by one).
78. If all m characters match, record position i as an occurrence.
79. Display all occurrence positions and the total comparisons.
80. Stop.

Python Program

```
def brute_force_match(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    comparisons = 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons

text = "AABAACAADAABAABA"
pattern = "AABA"
positions, comparisons = brute_force_match(text, pattern)
print("Pattern found at positions:", positions)
print("Total number of comparisons:", comparisons)
```

Input

```
text = "AABAACAADAABAABA"
pattern = "AABA"
```

Output

```
Pattern found at positions: [0, 9, 12]
Total number of comparisons: 30
```

Result

The brute force method checks every alignment of the pattern against the text, correctly finding the pattern at positions 0, 9 and 12 using 30 character comparisons - consistent with the $O(n \times m)$ worst-case bound.

Q12. Find All Occurrences of a Pattern

Aim

To find all occurrences of a pattern within a text using Brute Force String Matching.

Algorithm

81. Start.
82. Read the text and the pattern.
83. For each alignment position i from 0 to $n-m$: compare characters of the pattern with the corresponding text substring.
84. If every character matches, record position i as an occurrence.
85. If a mismatch occurs, move to the next alignment (shift by one position).
86. Continue until all alignments have been checked.
87. Display all occurrence positions found.
88. Stop.

Python Program

```
def brute_force_match(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            positions.append(i)
    return positions

text = "BANANABANANA"
pattern = "ANA"
positions = brute_force_match(text, pattern)
print("Occurrences at positions", ", ".join(map(str, positions)))
```

Input

```
text = "BANANABANANA"
pattern = "ANA"
```

Output

Occurrences at positions 1, 3, 7, 9

Result

The pattern "ANA" occurs 4 times in "BANANABANANA" (including overlapping occurrences at positions 1 and 3), confirming that brute force matching correctly detects overlapping pattern occurrences, unlike some optimized algorithms that skip past them.

Q13. Comparison Table for Each Alignment

Aim

To implement Brute Force String Matching and display a comparison table showing the shift number, comparisons made and match/mismatch result for each alignment.

Algorithm

89. Start.
90. Read the text and the pattern.
91. For each shift i from 0 to $n-m$: initialize a per-shift comparison counter.
92. Compare pattern characters against the text substring starting at i , counting comparisons.
93. Record whether the alignment resulted in a match or mismatch.
94. Print the shift number, comparisons made for that shift and the match/mismatch result.
95. Stop.

Python Program

```
def bf_comparison_table(text, pattern):
    n, m = len(text), len(pattern)
    table = []
    for i in range(n - m + 1):
        cmp_count = 0
        match = True
        for j in range(m):
            cmp_count += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        table.append((i, cmp_count, "Match" if match else "Mismatch"))
    return table

text = "MISSISSIPPI"
pattern = "ISSI"
for shift, cmp_count, result in bf_comparison_table(text, pattern):
    print(f"Shift {shift}: comparisons = {cmp_count}, result = {result}")
```

Input

```
text = "MISSISSIPPI"
pattern = "ISSI"
```

Output

```
Shift 0: comparisons = 1, result = Mismatch
Shift 1: comparisons = 4, result = Match
Shift 2: comparisons = 1, result = Mismatch
Shift 3: comparisons = 1, result = Mismatch
Shift 4: comparisons = 4, result = Match
Shift 5: comparisons = 1, result = Mismatch
Shift 6: comparisons = 1, result = Mismatch
Shift 7: comparisons = 1, result = Mismatch
```

Result

The pattern "ISSI" matches at shifts 1 and 4 (total 15 comparisons across all 8 shifts), while most mismatching shifts are rejected after just a single comparison, showing brute force's typical best-case behaviour of $O(1)$ per failed alignment on non-repetitive text.

Q14. Total Comparisons, Matches and Mismatches

Aim

To count the total character comparisons, total character matches and total character mismatches during Brute Force String Matching.

Algorithm

96. Start.
97. Read the text and the pattern.
98. For each alignment position i: compare pattern characters with the text substring one by one.
99. For every comparison, increment the comparison counter; increment matches on equality, else increment mismatches and break out of the inner loop.
100. If all characters match, record the alignment position as an occurrence.
101. After all alignments are processed, display total comparisons, matches and mismatches.
102. Stop.

Python Program

```
def bf_stats(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = matches = mismatches = 0
    positions = []
    for i in range(n - m + 1):
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                mismatches += 1
                break
            else:
                matches += 1
        else:
            positions.append(i)
    return positions, comparisons, matches, mismatches

text = "ABABABABAB"
pattern = "ABAB"
positions, comparisons, matches, mismatches = bf_stats(text, pattern)
print("Occurrences at:", positions)
print(f"Total comparisons = {comparisons}")
print(f"Total matches = {matches}")
print(f"Total mismatches = {mismatches}")
```

Input

```
text = "ABABABABAB"
pattern = "ABAB"
```

Output

```
Occurrences at: [0, 2, 4, 6]
Total comparisons = 19
Total matches = 16
Total mismatches = 3
```

Result

The highly repetitive text causes the pattern to match at every even shift (0, 2, 4, 6), producing many more matching comparisons (16) than mismatches (3) — an example where brute force approaches its $O(n \times m)$ worst-case behaviour due to the text's repetitive structure.

Q15. Best, Average and Worst Case Determination

Aim

To implement Brute Force String Matching on a worst-case-inducing input and determine whether the search exhibits best-case, average-case or worst-case behaviour.

Algorithm

103. Start.
104. Read the text and the pattern.
105. For each alignment position i from 0 to $n-m$: compare pattern characters against the text substring, counting comparisons.
106. Record the alignment as a match if all characters agree.
107. After processing all alignments, compare the total comparisons against the theoretical minimum $O(n)$ and maximum $O(n \times m)$ bounds.
108. Classify the run as best, average or worst case based on how close the count is to $n \times m$.
109. Stop.

Python Program

```
def bf_stats(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    positions = []
    for i in range(n - m + 1):
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
        else:
            positions.append(i)
    return positions, comparisons

text = "AAAAAAAAAB"
pattern = "AAAAB"
positions, comparisons = bf_stats(text, pattern)
n, m = len(text), len(pattern)
print("Occurrences at:", positions)
print(f"Comparisons performed = {comparisons} (theoretical worst case =  $n \times m = \{n \times m\}$ )")
```

Input

```
text = "AAAAAAAAAB"
pattern = "AAAAB"
```

Output

```
Occurrences at: [5]
Comparisons performed = 30 (theoretical worst case =  $n \times m = 50$ )
```

Result

Because the text is almost entirely the repeated character 'A', most alignments compare several characters before finally mismatching on the last character of the pattern, making this close to the worst-case scenario for brute force matching ($O(n \times m)$), even though the absolute count (30) is below the full 50 since the final alignment succeeds outright.

Q16. First Occurrence Only

Aim

To search for a pattern in a text using Brute Force String Matching and report only the first occurrence, along with the number of comparisons used.

Algorithm

110. Start.
111. Read the text and the pattern.
112. For each alignment position i from 0 to $n-m$: compare pattern characters against the text substring, counting comparisons.
113. If all characters match, return this position immediately as the first occurrence, along with the comparisons made so far.
114. If a mismatch occurs, move to the next alignment ($i+1$).
115. If no alignment matches after scanning the whole text, report 'Pattern not found'.
116. Stop.

Python Program

```
def first_occurrence_bf(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            return i, comparisons
    return -1, comparisons

text = "COMPUTERSCIENCE"
pattern = "SCI"
position, comparisons = first_occurrence_bf(text, pattern)
print(f"First occurrence position = {position}")
print(f"Number of comparisons = {comparisons}")
```

Input

```
text = "COMPUTERSCIENCE"
pattern = "SCI"
```

Output

```
First occurrence position = 8
Number of comparisons = 11
```

Result

The search stops as soon as the first match is found at position 8, needing only 11 comparisons rather than continuing to check every remaining alignment in the text - useful when only the first match matters.

Q17. Case-Insensitive Brute Force String Matching

Aim

To implement Brute Force String Matching that ignores letter case while searching for a pattern in a text.

Algorithm

117. Start.
118. Read the text and the pattern.
119. Convert both the text and the pattern to a common case (uppercase).
120. Apply the standard brute force matching procedure on the converted strings.
121. Report the position where the pattern is found (relative to the original text).
122. If no match is found, report 'Pattern not found'.
123. Stop.

Python Program

```
def first_occurrence_bf(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            return i
    return -1

def bf_case_insensitive(text, pattern):
    return first_occurrence_bf(text.upper(), pattern.upper())

text = "DataStructuresAndAlgorithms"
pattern = "ALGORITHMS"
position = bf_case_insensitive(text, pattern)
print(f"Pattern found at position {position}")
```

Input

```
text = "DataStructuresAndAlgorithms"
pattern = "ALGORITHMS"
```

Output

Pattern found at position 17

Result

By normalizing both strings to uppercase before matching, the algorithm correctly locates "ALGORITHMS" within the mixed-case text "...AndAlgorithms" at position 17, without altering the underlying $O(n \times m)$ brute force comparison logic.

Q18. Successful vs Unsuccessful Search Comparison

Aim

To compare the execution of Brute Force String Matching for a successful search and an unsuccessful search on the same text.

Algorithm

124. Start.
125. Read the text and two patterns: one expected to be present, one expected to be absent.
126. For pattern 1: run brute force matching, counting comparisons until a match is found.
127. For pattern 2: run brute force matching over the entire text, counting comparisons since no match will be found.
128. Display the comparison counts for both cases.
129. Compare and interpret the difference between successful and unsuccessful search costs.
130. Stop.

Python Program

```
def first_occurrence_bf(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            return i, comparisons
    return -1, comparisons

text = "PROGRAMMINGLAB"
pattern1 = "LAB"
pattern2 = "TEST"
pos1, cmp1 = first_occurrence_bf(text, pattern1)
pos2, cmp2 = first_occurrence_bf(text, pattern2)
print(f'Successful search ('LAB'): position = {pos1}, comparisons = {cmp1}')
print(f'Unsuccessful search ('TEST'): position = {pos2}, comparisons = {cmp2}')
```

Input

```
text = "PROGRAMMINGLAB"
pattern1 = "LAB"
pattern2 = "TEST"
```

Output

```
Successful search ('LAB'): position = 11, comparisons = 14
Unsuccessful search ('TEST'): position = -1, comparisons = 11
```

Result

The successful search for "LAB" needed 14 comparisons to confirm the match at the very end of the text, while the unsuccessful search for "TEST" needed only 11 comparisons because most alignments mismatched immediately on the first character - showing that an unsuccessful search is not always more expensive than a successful one.

Q19. Display Every Alignment of the Pattern

Aim

To implement Brute Force String Matching and display every alignment number along with its matching result and the final occurrence positions.

Algorithm

- 131.Start.
- 132.Read the text and the pattern.
- 133.For each alignment number i from 0 to $n-m$: compare the pattern against the text substring at that alignment.
- 134.Record whether the alignment resulted in a match or mismatch.
- 135.If it is a match, add the alignment position to the occurrence list.
- 136.After checking all alignments, display the full alignment log and the final list of occurrence positions.
- 137.Stop.

Python Program

```
def bf_all_alignments(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    log = []
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            if text[i + j] != pattern[j]:
                match = False
                break
        log.append((i, "Match" if match else "Mismatch"))
        if match:
            positions.append(i)
    return log, positions

text = "ABCDABCABCD"
pattern = "ABCD"
log, positions = bf_all_alignments(text, pattern)
for alignment, result in log:
    print(f"Alignment {alignment}: {result}")
print("Pattern occurrence positions:", positions)
```

Input

```
text = "ABCDABCABCD"
pattern = "ABCD"
```

Output

```
Alignment 0: Match
Alignment 1: Mismatch
Alignment 2: Mismatch
Alignment 3: Mismatch
Alignment 4: Mismatch
Alignment 5: Mismatch
Alignment 6: Mismatch
Alignment 7: Match
Pattern occurrence positions: [0, 7]
```

Result

The pattern "ABCD" occurs at alignments 0 and 7 out of the 8 possible alignments; displaying every alignment's result makes explicit how brute force matching tries and discards each shift independently, with no memory of prior partial matches.

Q20. Complete Program with Complexity Analysis

Aim

To write a complete Brute Force String Matching program that finds all occurrences of a pattern, counts total comparisons, displays the shifting process, and analyzes best-case, worst-case and space complexity.

Algorithm

138. Start.
139. Read the text (length n) and the pattern (length m).
140. For each shift i from 0 to $n-m$: compare pattern characters against $\text{text}[i..i+m-1]$, counting comparisons.
141. Display the shift number and whether it produced a match or mismatch.
142. Record i as an occurrence position whenever all m characters match.
143. After all shifts are processed, display all occurrence positions and the total comparison count.
144. Compare the comparison count against the theoretical best-case $O(n)$ and worst-case $O(n \times m)$ bounds; state that no extra data structures are used ($O(1)$ auxiliary space beyond the position list).
145. Stop.

Python Program

```
def brute_force_full(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    comparisons = 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        print(f"Shift {i}: {'Match' if match else 'Mismatch'}")
        if match:
            positions.append(i)
    return positions, comparisons

text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
pattern = "TATCTT"
positions, comparisons = brute_force_full(text, pattern)
n, m = len(text), len(pattern)
print("Occurrence positions:", positions)
print(f"Total comparisons = {comparisons}")
print(f"Worst-case bound  $n*m = \{n*m\}$ , Best-case bound =  $n = \{n\}$ ")
```

Input

```
text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT" (n = 38)
pattern = "TATCTT" (m = 6)
```

Output

```
Shift-by-shift Match/Mismatch log for all 33 shifts (0 to 32)
Occurrence positions: [] (pattern does not occur in this text)
Total comparisons = 57
Worst-case bound  $n*m = 228$ , Best-case bound =  $n = 38$ 
```


Result

No occurrence of "TATCTT" exists in the given text, so the search is unsuccessful; it still needs only 57 of the 228 worst-case comparisons because most alignments mismatch within the first couple of characters.

Best-case complexity is $O(n)$ (immediate mismatch at every shift), worst-case complexity is $O(n \times m)$ (pattern almost matches at every shift), and space complexity is $O(1)$ auxiliary space (excluding the input and output lists).